

Búsqueda de factores RSA compartidos en los Registros de Transparencia de Certificados

un enfoque basado en Binary Tree Batch GCD

David Quinzaños Saiz, Ana I. Gómez Pérez, Domingo Gómez Pérez
Departamento de Matemáticas, Estadística y Computación — Universidad de Cantabria

Trabajo en curso
Resultados finales
en Altamira pendientes

Motivación y objetivo

La seguridad de RSA depende de que los primos usados para construir cada módulo $N = p \cdot q$ sean impredecibles y no se reutilicen. Si dos módulos comparten un factor primo, basta calcular $\gcd(N_i, N_j)$ para comprometer ambas claves.

Este escenario es especialmente relevante en certificados digitales, porque la clave pública RSA forma parte del certificado y puede obtenerse de forma abierta. Los Registros de Transparencia de Certificados (*Certificate Transparency logs*) proporcionan, por tanto, una fuente masiva de claves reales emitidas en Internet sobre la que es posible realizar auditorías criptográficas.

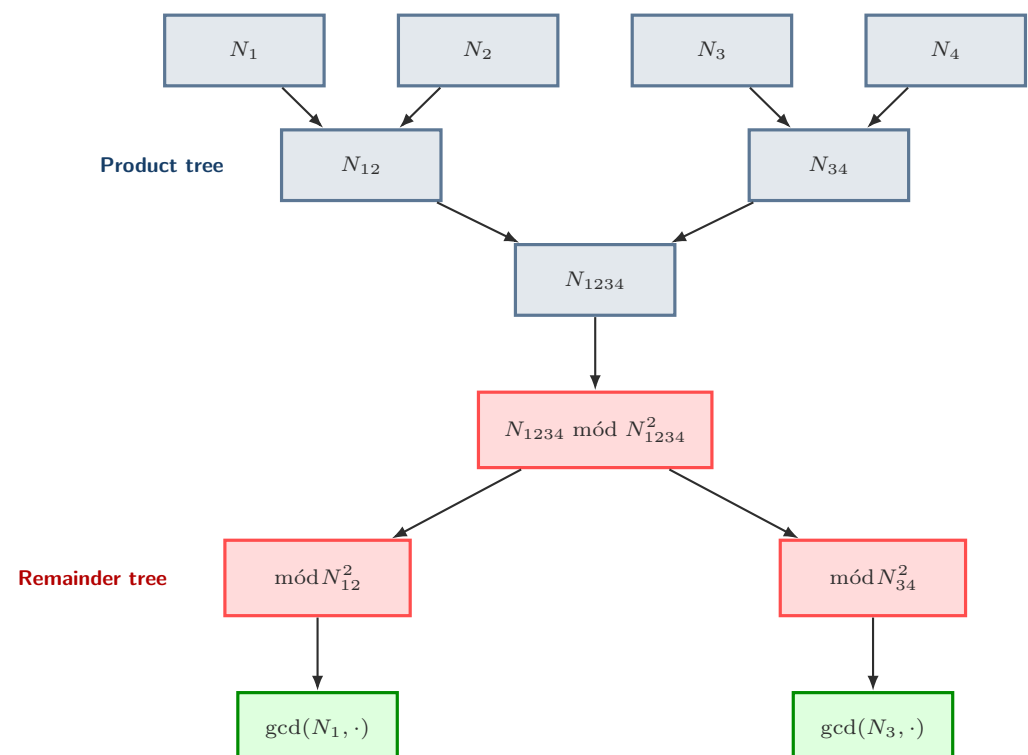
Este problema ya fue estudiado en la tesis *Finding shared RSA factors in the Certificate Transparency logs*, donde se analizan certificados reales extraídos de CT logs mediante técnicas *batch GCD* clásicas. Ese trabajo demuestra el interés práctico de auditar los CT logs, pero también evidencia el elevado coste computacional del análisis masivo.

Como mejora algorítmica, el artículo de Pelofske propone *Binary Tree Batch GCD*, un método más eficiente que evita parte del coste del enfoque clásico basado en *remainder tree*.

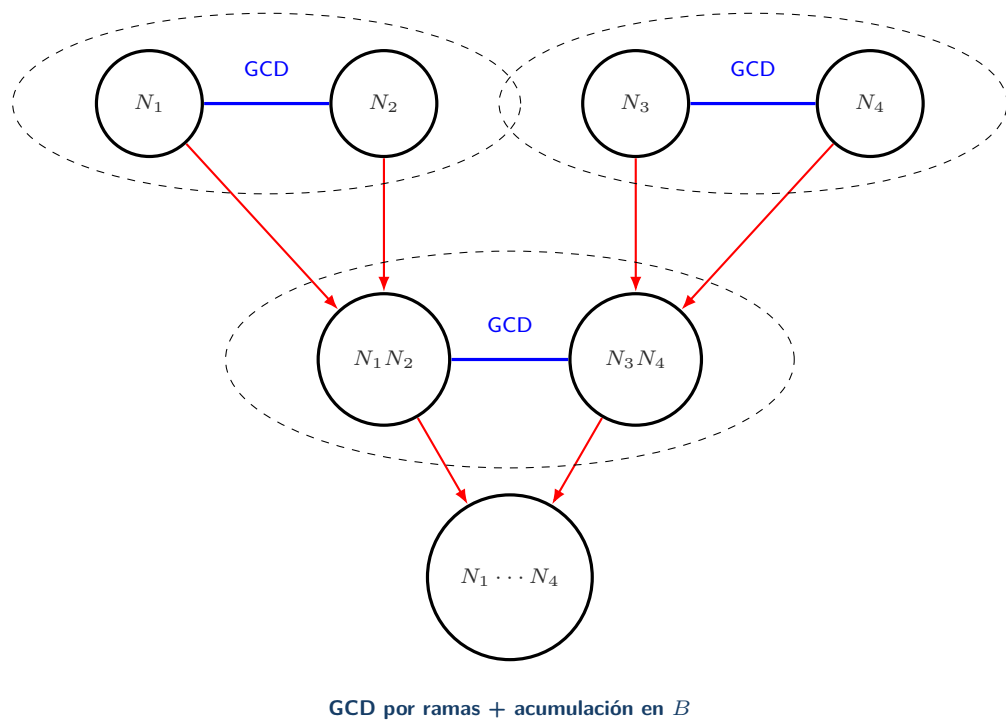
Este TFM conecta ambas líneas: toma como objetivo aplicado la búsqueda de factores RSA compartidos en CT logs, evalúa el algoritmo *Binary Tree Batch GCD* frente al método clásico y desarrolla una implementación C++17/GMP/OpenMP preparada para ejecución masiva en Altamira.

Algoritmo e implementación

El enfoque ingenuo requiere $\mathcal{O}(M^2)$ cálculos GCD para M módulos RSA. El *batch GCD* clásico evita esta comparación par a par mediante un *product tree* y un *remainder tree*, pero esta segunda estructura tiene un coste elevado.



Binary Tree Batch GCD elimina el *remainder tree* completo: durante el árbol de productos calcula GCD entre los hijos de cada nodo, agrega los GCD no triviales en una variable B y finalmente comprueba $\gcd(N_i, B)$. Esta estrategia es más eficiente cuando los factores compartidos son pocos respecto al número total de claves, como se espera en CT logs reales.



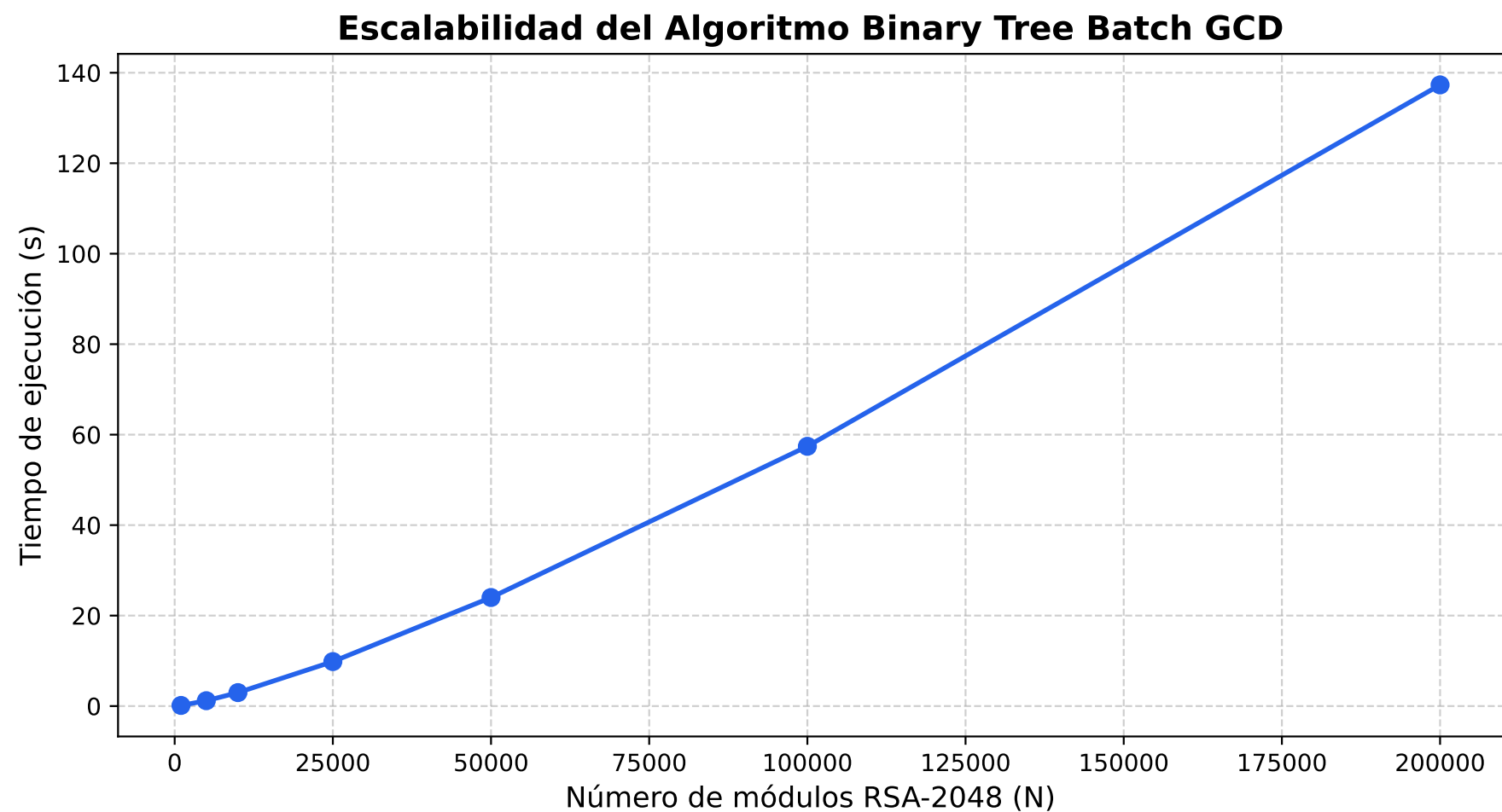
Las versiones Python de Pelofske se usan como referencia, pero su sobrecarga en enteros grandes limita el escalado. Por ello se implementa el algoritmo en C++17 con GMP y OpenMP, mejorando la aritmética multiprecisión, el control de memoria y la paralelización.

La validación combina dos fases: primero experimentos sintéticos en local, con módulos RSA de 1024 y 2048 bits y distintos números de claves débiles; después, ejecución en Altamira para reproducir el benchmark con los tamaños originales del artículo y atacar el conjunto real de certificados de CT logs.

Resultados preliminares

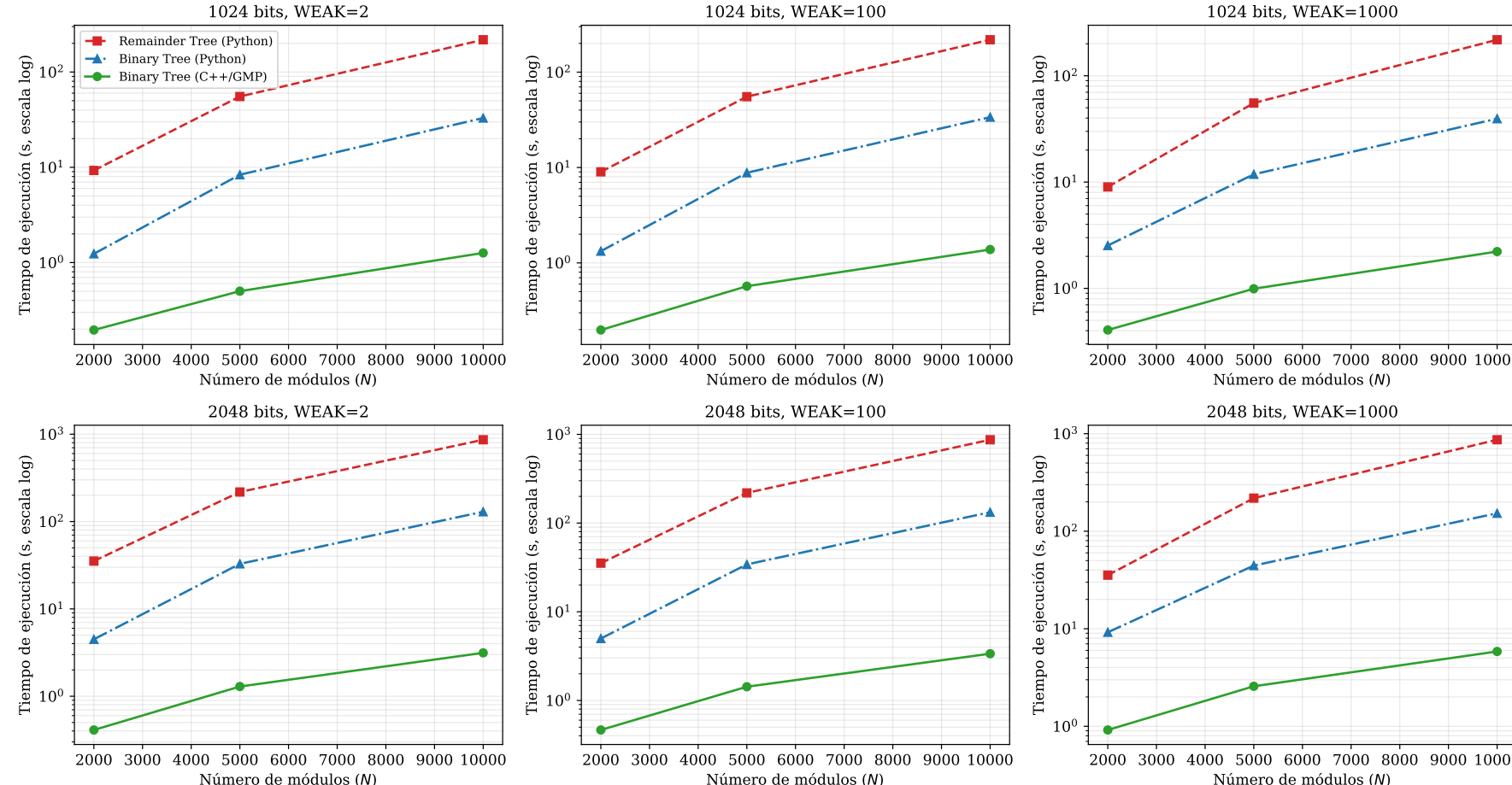
El procedimiento experimental se planteó de forma incremental. Primero se desarrolló una implementación propia de *Binary Tree Batch GCD* en C++/GMP, con el objetivo de disponer de una versión más eficiente que las implementaciones Python y adecuada para una ejecución posterior sobre datos reales.

A continuación se generaron datos sintéticos con módulos RSA controlados y un número conocido de claves débiles. Esta fase permitió comprobar que el algoritmo detectaba correctamente los factores compartidos y estudiar cómo variaba el tiempo de ejecución al aumentar el número de módulos.



Una vez validada la implementación propia, se utilizó el repositorio asociado al artículo de Pelofske para reproducir una comparación más próxima al experimento original. El repositorio aportaba las implementaciones Python de *Remainder Tree Batch GCD* y *Binary Tree Batch GCD*, además de parte de los primos usados para generar los módulos sintéticos.

Para integrarlo en el TFM fue necesario adaptar el código original: corregir incompatibilidades con versiones actuales de Python y NumPy, crear *wrappers* de ejecución, normalizar la entrada de módulos y generar los primos adicionales necesarios para evaluar también módulos RSA de 2048 bits.



Con ello se compararon los tres algoritmos sobre los mismos datos de entrada: el método clásico en Python, el algoritmo de Pelofske en Python y la implementación propia en C++/GMP, usando módulos RSA de 1024 y 2048 bits y distintos números de claves débiles.

Acercación observada

En promedio, *Binary Tree* Python fue aproximadamente $6,1\times$ más rápido que *Remainder Tree* Python. La implementación C++/GMP fue aproximadamente $116,5\times$ más rápida que *Remainder Tree* Python y $18,7\times$ más rápida que *Binary Tree* Python.

El caso más exigente de la parrilla local corresponde a módulos RSA-2048, $N = 10000$ y WEAK=1000: el método clásico tardó 866.819 s, la versión Python del algoritmo de Pelofske 152.803 s y la implementación C++/GMP 5.851 s.

Bits	WEAK	N	Remainder Py.	Binary Py.	Binary C++
1024	100	10000	218.579 s	33.721 s	1.381 s
2048	100	10000	870.017 s	132.484 s	3.368 s
2048	1000	10000	866.819 s	152.803 s	5.851 s

La implementación C++/GMP alcanza hasta $275\times$ de aceleración frente al método clásico en Python.

Validación con datos reales

Sobre una muestra local de certificados ya extraída y normalizada:

Elemento	Valor
Registros de certificado	112230
Registros con módulo RSA	87462
Registros sin módulo RSA	24768
Módulos RSA únicos	601
RSA-2048 únicos	318
RSA-3072 únicos	94
RSA-4096 únicos	189

La ejecución local sobre 601 módulos únicos no detectó factores compartidos y completó el análisis en 107.17 ms. Este resultado no pretende ser concluyente sobre la seguridad de los CT logs: su función es validar que el flujo completo funciona sobre datos reales.

El conjunto local es pequeño frente a los estudios previos y frente al objetivo final en Altamira. Su utilidad es comprobar que los módulos reales llegan al binario C++ en el formato correcto y que la deduplicación previa evita trabajo redundante.

Ejecución pendiente en Altamira

El resultado final del póster se sustituirá por la campaña en Altamira:

- benchmark completo con las N originales del artículo;
- ataque distribuido sobre el conjunto completo de certificados de CT logs;
- registro de tiempos, memoria pico, módulos RSA válidos, módulos únicos y factores compartidos encontrados.

El diseño permite lanzar trabajos por bloques con un parámetro N modificable, manteniendo el ataque global correcto mediante una comparación de cada bloque contra el producto global de todos los módulos.

Esta separación permite explotar varios nodos sin convertir el análisis en una comparación parcial por bloques: si dos claves comparten un primo aunque estén en trabajos distintos, el método distribuido sigue pudiendo detectarlo.

Estado actual

- Benchmark local validado frente a las implementaciones Python originales.
- Implementación C++/GMP preparada para ejecución HPC.
- Pipeline real validado: extracción, filtrado, deduplicación y análisis.
- Pendiente: sustituir resultados preliminares por la salida completa de Altamira.

La versión actual del póster usa resultados locales como marcador técnico. La estructura está preparada para reemplazar la tabla y las cifras por los CSV de Altamira cuando finalicen las ejecuciones.

Conclusiones provisionales

- *Binary Tree Batch GCD* reduce el coste práctico frente al método clásico basado en *remainder tree*.
- La implementación C++17/GMP mejora de forma clara a las versiones Python del repositorio original.
- El flujo completo de extracción, normalización y análisis se ha validado sobre certificados reales.
- Altamira permitirá reemplazar los resultados provisionales por una evaluación a escala real.

El trabajo ya valida metodología e implementación. La contribución final será cuantificar el comportamiento a escala y, si aparecen, reportar los factores RSA compartidos encontrados en el conjunto completo de CT logs.

Referencias

- R. L. Rivest, A. Shamir, L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems", *Communications of the ACM*, 1978.
- N. Heninger et al., "Mining Your Ps and Qs", *USENIX Security*, 2012.
- H. F. Våge, "Finding shared RSA factors in the Certificate Transparency logs", Master's Thesis, University of Bergen, 2022.
- E. Pelofske, "An Efficient All-to-All GCD Algorithm for Low Entropy RSA Key Factorization", Preprint, 2024.